What's new

DHJ: Writing the second paper

Terence Tao · 22 May 2009

https://terrytao.wordpress.com/2009/05/22/dhj-writing-the-second-paper/

tao2typ · converted to Typst 2026-10-14 (UTC)

Now that the quarter is nearing an end, I'm returning to the half of the polymath1 project hosted here, which focussed on computing density Hales-Jewett numbers and related quantities. The purpose of this thread is to try to organise the task of actually writing up the results that we already have; as this is a metathread, I don't think we need to number the comments as in the research threads.

To start the ball rolling, I have put up a proposed outline of the paper on the wiki. At present, everything in there is negotiable: title, abstract, introduction, and choice and ordering of sections. I suppose we could start by trying to get some consensus as to what should or should not go into this paper, how to organise it, what notational conventions to use, whether the paper is too big or too small, and so forth. Once there is some reasonable consensus, I will try creating some TeX files for the individual sections (much as is already being done with the first polymath1 paper) and get different contributors working on different sections (presumably we will be able to coordinate all this through this thread). This, like everything else in the polymath1 project, will be an experiment, with the rules made up as we go along; presumably once we get started it will become clearer what kind of collaborative writing frameworks work well, and which ones do not.

Comments (110)



Terence Tao · 13 June 2009

Hmm. The only way that a 6D set can have statistic (24,72,180,80,0,0,0) is if every 1***** slice has statistics (6,12,18,4,0) (which is an extremal for 4D), and similarly for permutations. If we can classify the (6,12,18,4,0) sets in [3]^4 we may be able to kill off the 356 case. The scanning program can in principle do this but perhaps an integer program would be the most direct way to go here?



Terence Tao · 13 June 2009

Ack, there is a problem with the brute force search :-(... (6,12,18,4,0) is reported as feasible, but it is not compatible with the 3D inequalities. Time to debug...



Terence Tao · 13 June 2009

False alarm! (6,12,18,4,0) is indeed feasible (an explicit example has slices 261660105, 356516660,432356261 in octal; it seems that there are not many other solutions than this one and its symmetric permutations, though I was not able to establish this rigorously). My application of the 3D inequalities in the preceding comment was incorrect. So, barring any further bugs, we're still in a good position, in that we have a proof of $353 \leq c'_6 \leq 356$ and a fairly precise description of when $c'_6 = 356$ can occur; if we can pin down the (6,12,18,4,0) Moser sets precisely, we might be able to finish off 356 completely.



Kristal Cantwell · 13 June 2009

If the slice is 4D from a 6D set shouldn't the number of fixed coordinates be two instead of one. I am assuming that what happens here is that is that if we fix any two coordinates of the (24,72,180,80,0,0,0) and set each of them equal to 1 or 3 the resulting slice will have statistics (6,12,18,4,0).



Terence Tao · 13 June 2009

Yes, you're right, I meant the 11**** slices (and symmetries thereof).

Actually I have a short proof of $c'_6 \leq 356$. Define the *score* of a 4D set to be 4a+6b+10c+20d+60e. Double counting shows that (if f=g=0, which we may assume) the size of a 6D set is the average of the scores of its 4D side slices (such as 11****). But thanks to the 4D extremals, the largest score is 356, attained at exactly one place, namely (6,12,18,4,0), see https://spreadsheets.google.com/ccc?key=rD2Oc_wyheVOcmCyd-Drdgw&hl=en . This also shows that the only way 356 is attainable is if all 11**** slices have statistics (6,12,18,4,0) as mentioned earlier, and if the entire set has statistics (24,72,180,80,0,0,0).

With a bit more effort I can get $c'_6 \leq 355$. Consider the 112*** slices (and symmetries thereof). Double counting shows that these slices must have statistics (3,9,3,0) on the average. But all 3D slices must obey the inequalities $b + c + 3d \leq 12$ and $3a + 2b + 3c + 6d \leq 36$ (see https://spreadsheets.google.com/ccc?key=p5T0SktZY9DuKZ2DyzO9EOg&hl=en), and so all 112*** slices must obey these inequalities with equality; also, d must be zero. We conclude that the 112*** slices have statistics (3,9,3,0), (2,6,6,0), or (4,12,0,0). The latter two possibilities are inconsistent with the 11**** slices having statistics (6,12,18,4,0) (since all but at most two of the "d" points in the 11**** slice lie in 112***). So all 112*** slices have statistics (3,9,3,0). Since 11**** has statistics (6,12,18,4,0), this implies that exactly one of 111222 and 113222 lie in the set. More generally, we see that given any two adjacent "d" points in $[3]^6$, exactly one of them lies in the set; thus the d points consist either of those strings with an even number of 1s, or those with an odd number of 1s.

Let's say it's the former, thus the set contains 111222, 133222, and permutations, but omits 113222, 333222 and permutations. Meanwhile, we see that the (24,72,180,80,0,0,0) set saturates the inequality $4c + 3d \leq 960$, which is only possible if every line connecting two "c" points to a "d" point meets exactly two elements in the set. Since the "d" points 113222, 333222 are omitted, we conclude that the "c" points 113122, 113322, 333122, 333322 must lie in the set, and similarly for permutations. But this is too many c points (225 points, whereas we are only supposed to have 180), a contradiction.

There may be a chance to push these arguments further; the fact that (6,18,24,4,0) is the only extremiser for the score (by quite a margin) is quite encouraging.



Michael Peake · 14 June 2009

I can see why, in xxx222, either every d point has an odd number of 1s, or an even number of 1s. But xx2x22 may also have an odd number of 1s, or an even number of 1s. Could a choice of 'odd' and 'even' for the twenty classes of d points be made so that fewer c points are needed?

 **Terence Tao** · 14 June 2009

Hmm, true, but the argument can be easily fixed. For xxx222, the argument in my previous comment shows that the set has at least 15 "c" points of the form xxxx22 (regardless of whether the xxx222 points come from odds or evens). Permuting this fact, we still get 225 "c" points in all.

I played a little bit with the score 4a+6b+10c+20d+60e. As I said before, the maximal score is 356, attained at (6,18,24,0,0). The next highest score among all feasible statistics is 352, attained in three ways, by (6,8,12,8,0), (5,12,12,4,1), and (5,18,24,0,0). Since there must be a way to slice into 11xxxx, 13xxxx, 31xxxx, 33xxxx (or a permutation thereof) such that the average score is greater than or equal to the cardinality, we see that for a 355 set, one way of slicing must consist of three (6,18,24,0,0) slices plus one of the three other slices mentioned above (in particular, this shows that a=24 or a=23). To proceed further it seems of interest to classify the (6,18,24,0,0) slices properly. It would also be interesting to see what the slices of Kareem's example are.

 **DHJ: Still writing the second paper « What's new** · 14 June 2009

[...] June, 2009 in math.CO | Tags: polymath1 | by Terence Tao This is a continuation of the previous thread here in the polymath1 project, which is now full. Ostensibly, the purpose of this thread is to [...]

 **Terence Tao** · 14 June 2009

It's a little awkward to move the thread while it is still so active, but I have decided to stick with tradition and move on to the next thread, at

> DHJ: Still writing the second paper

 **DHJ: Writing the second paper III. « What's new** · 9 July 2009

[...] July, 2009 in math.CO | Tags: polymath1 | by Terence Tao This is a continuation of the preceding two threads here on the polymath1 project, which are now full. We are closing on on the [...]

 **Terence Tao** · 6 June 2009

Dear Michael, this is promising, it looks like the algorithm run time is almost within a feasible range, especially since it should be possible to split up the task among several computers (e.g. to do the case with 2222 and the case without 2222 separately). Can I ask how you search for the Pareto maxima for each forbidden set? It may be possible to squeeze an extra factor of 2 or something by an additional trick. (In particular, I suspect that the with-2222 and without-2222 cases each take less than half of the projected time of the whole algorithm, due to additional shortcuts one can take in either case (e.g. filling in the 2222 point greedily)).

The one additional idea that occurred to me is that if one already knew the Pareto optimal statistics for one forbidden set, then this would give partial information about the statistics for any larger forbidden set (and also for any smaller forbidden set), which may reduce the number of statistics one has to search. Given that it's only taking half a second now to check

each forbidden set, though, the benefit of trying to set this up may not be worthwhile. (It also interacts somewhat clunkily with the 48 symmetries.)

 **Kristal Cantwell** · 6 June 2009

I have four more cases. I now have 12/16 of them. All of them are at the end of the section n=3 at

http://michaelnielsen.org/polymath1/index.php?title=Moser%27s_cube_problem

 **Klas Markström** · 7 June 2009

Michael, is your final program in Matlab too? I have several machines where I can run programs but they do not have Matlab since they moslty for batch work.

 **Michael Peake** · 7 June 2009

Klas,

My final program is in Matlab; I haven't worked out how to compile and run Matlab routines outside of Matlab. I put the program in the Moser wiki page. I have Student Matlab 5, which is a little out of date.

Note this routine is only to build the lookup table. For each subset of [3]^3, it gives a list of Pareto-max statistics, of line-free sets restricted to that subset.

The routine is easily parallelizable. Speedups would be welcome as, at an average of half a second per restriction subset, it would take two weeks to run.

 **Kristal Cantwell** · 7 June 2009

I have finished all sixteen cases. The are at the end of the section n=3 of the Moser wiki.

 **Klas Markström** · 7 June 2009

Michael,
If the program can be run in command line Matlab I have available at least 16 linux machines where I can run it if you want to cut the final run time down.

For things which are pure C, or something similar, I have a lot more available.

 **Terence Tao** · 7 June 2009

Kristal: That's great! I'll try to go through them at some point. They may either go into an appendix of the paper, or else we could leave them on the wiki and put a link to them from the paper.

Michael: A couple speedups occurred to me. Firstly, once we compute the Pareto statistics (a,b,c,d) for forbidden sets avoiding 222, then we automatically get the Pareto statistics for forbidden sets with 222 as well – just set d to zero! So we need to only check 2^{26}/48 configurations rather than 2^{27}/48.

Once 222 is no longer forbidden, we can fill in 222 greedily among the 3813884 Moser sets, which may cut down the possibilities a little bit.

Another observation is that the (1,b,c,d) statistics can be deduced from the (0,b,c,d) statistics, because if one has a (0,b,c,d) Moser set, one can automatically create a (1,b,c,d) Moser set by

adding an arbitrary a-point. (It requires two a-points to make a line.) So unless all 8 a-points are forbidden, we can move a from 0 to 1 "for free". Similarly we can deduce the (0,1,c,d) statistics from (0,0,c,d), and (0,0,1,d) from (0,0,0,d). This trims down the cases a little bit (this is a pretty modest saving though.)

In a similar spirit, it is easy to see when (a,0,0,0) is feasible – this occurs whenever at least a of the a-points are not forbidden. Similarly for (0,b,0,0), (0,0,c,0), (0,0,0,d). With all this we should be able to reduce the 499 possible (a,b,c,d) statistics for Moser sets to a modestly smaller number.



Terence Tao · 7 June 2009

OK, I dusted off my old C programming skills and managed to get an algorithm that tests each forbidden set in about 0.07 seconds on my laptop, which when multiplied by $2^{26}/48$ would become roughly a day (but perhaps less, on a dedicated computer).

What I did was as follows. I brute-force located all 3813884 3D Moser sets (confirming Michael's count – actually, it only takes about 15 seconds to find them all), and sorted them into the 499 (a,b,c,d) possible statistics, creating 499 lists. I then threw away the redundant or easy statistics from my previous comment (i.e. (1,b,c,d), (0,1,c,d), (0,0,1,d), (a,0,0,0), (0,b,0,0), (0,0,c,0), (0,0,0,d)), leaving 369 lists, totaling 3151007 sets in all. (Full statistics can be found here..) I sorted these lists by size; most are quite small, for instance 159 of them are under 1000 in length, the longest being (2,5,2,0) which has 108528 elements. To find the statistics for each choice of forbidden set, I then started with the smallest lists first and worked down towards the largest lists, but each time I found a feasible set (a,b,c,d), I then skipped all smaller choices of (a,b,c,d), and conversely when I found that (a,b,c,d) was infeasible I skipped all larger choices of (a,b,c,d). Also, I initially computed the maximal values of a,b,c,d compatible with the forbidden set and excluded any (a,b,c,d) that were not less than or equal to these maximal values.

Michael, just to double check my code, could you plug in the following randomly chosen forbidden sets (written as 9 octal digits = 27 bits, identified with subsets of $[3]^3$ by identifying the latter with {0,...,26} using base 3 – actually the beauty of the Moser set problem is that the conventions here really don't matter, due to the symmetries of the Moser problem) into your own code and see what Pareto-optimals you get? My Pareto-optimal list is likely to be a little bit shorter (avg length is about 5.5) because I removed all the redundant statistics. Alternatively, you could send me some of your randomly chosen lists so that I can check my own code...

405301460...(3 6 2 1) (3 5 3 1) (3 9 0 0) (3 6 4 0) (3 7 3 0) (4 7 1 0) (4 6 2 0) (3 8 1 0) (4 4 3 0)

607626507...(0 4 4 0) (0 6 2 0) (0 5 3 0)

340302444...(4 5 4 0) (3 6 2 1) (4 6 3 0) (4 7 2 0) (5 3 2 0) (5 5 0 0) (5 4 1 0) (3 8 1 0)

645570745...(0 6 3 0) (0 4 4 0)

If it checks out, then I'll try to write the rest of the code that I could give to Klas to run on a better machine than my laptop.

 **Michael Peake** · 7 June 2009

Terry,

Unfortunately, our Pareto lists don't match. Here are my results:

Oct(405301460)=Dec(68518704)
(3 5 1 1) (2 9 1 0) (3 8 2 0) (5 5 0 0) (4 7 1 0) (3 9 0 0)
Oct(607626507)=Dec(102706503)
(2 4 3 0) (1 6 1 0) (2 5 2 0) (1 5 2 1) (2 4 2 1)
Oct(340302444)=Dec(58819876)
(3 5 2 1) (3 6 2 0) (3 3 3 1) (3 4 4 0) (3 5 3 0) (4 4 2 0) (2 6 3 0) (2 5 3 1) (3 7 1 0) (4 5 1 0) (2 5 5 0)
Oct(645570745)=Dec(110555621)
(2 5 3 0) (3 4 3 0) (2 6 2 0)

I'll recheck my program.

 **Terence Tao** · 7 June 2009

Hmm, maybe we have different conventions? For instance, for the forbidden set Oct(405301460)=Dec(68518704), I get Oct(252436105)=Dec(44710981) as a Moser set with statistics (3,6,2,1) which is disjoint from the forbidden set, so it doesn't look like to me as if (3,6,1,1) can be Pareto-optimal.

Also, Oct(607626507)==Dec(102706503) contains the centre square 222 (because of the "2" in the middle of the octal string) and so the (1,5,2,1) and (2,4,2,1) statistics in your Pareto-optimal list can't be right.

Not sure what is going on here…

 **Michael Peake** · 8 June 2009

Yes, I found my bug. I multiplied by 256 when I should have multiplied by 512 to go from one layer to the next. Now my results agree with yours.

 **Terence Tao** · 8 June 2009

I managed to build a lookup table; it is 113MB (but a mere 643KB when compressed); I'll upload the latter when I get into work tomorrow. (It turned out that the 0.06 seconds / forbidden set runtime was dominated by the outputting of my debugging code (!); without it, the run time dropped by a factor of ten, and I was able to generate the table in a matter of minutes on my laptop. The (rather inelegant) code for generating the table is here.)

In order to make the lookup table fixed-width (to make it as easy to access as possible) I chose a rather idiosyncratic format. The 2^{26} forbidden sets (excluding 222) were split into a 12-bit string (representing the b's) and a 14-bit string (representing the a's and c's). The b-string was then mapped to the reflection of itself (among all 48 reflections) which had the smallest b-value; of the 4096 b-strings, there are 144 minimal such b-strings. So the total number of forbidden sets is 144 * 2^{14}.

For each forbidden set A, and each value of a,c,d, I recorded the largest b-value such that (a,b,c,d) is a feasible statistic of a Moser set avoiding A (or -1, if no such b-value exists). It turns out that the number of nontrivial values of (a,c,d) is 49, so I used 49 bytes per forbidden

set, for a total of 144 * 2^{14} * 49 = 110 MB. (Presumably the extra 3MB in my file are for file management purposes and checksums.)

In the next few days I'll test the lookup table by checking the test statistics given earlier, and then I should be able to build the outer loop to test the roughly 3813884^2 / (2 * 48) ~ 1.5 x 10^11 combinations. I haven't costed this yet, but this step may well require more computational power than my laptop has...

 **Klas Markström** · 8 June 2009

Terry, I'll be happy to supply some computing power if more is needed, and the program can be run on the machines I have available, which are mostly linux.

 **Michael Peake** · 9 June 2009

Here is a suggestion that uses the Pareto lists to do the full calculation:

* Pick an abcd1 statistic for Layer 1. Pick an abcd3 statistic for Layer 3.
* Initialize the list of possible Layer2 statistics
* For one representative per 48 with the abcd1 statistic, and for each pattern with the abcd3 statistic, find the forbidden set. Turn on the Pareto statistics for this forbidden set.
* Find the pareto optima of the possible Layer 2 statistics. Combine with abcd1 and abcd3, and compare with the [3]^4 pareto optima.
* Loop through other abcd1 and abcd2 statistics.

There is the following shortcut when 2222 is disallowed.
To find the forbidden set, you can group together all those Layer3s with the same a and b points, and same number of c points. Each forbidden Layer 2 set can be found from a common ancestor with a single bitand and bitor. This common ancestor also applies to different numbers of c points, so its calculation can be averaged over up to ninety Layer3 sets.

There are on average 5 or 6 pareto sets for each forbidden set, so the main part of the algorithm might take a dozen or so flops, times 83000*3800000/2

 **Terence Tao** · 9 June 2009

Dear Michael: Thanks for the suggestion! Certainly one wants to cut down the number of operations per 83000*3800000/2 Level 1/Level 3 pairs as much as possible. It requires loading an array of size 2^{27} = 128 MB for the possible Level2 statistics into memory (in addition to the 110MB lookup table I already have) but it seems that even my laptop will be capable of doing both simultaneously – I'm glad memory technology has advanced in the last 10 years! (I guess I could compress the table to 2^{27} bits rather than 2^{27} bytes, but this might cause a few more bitops per Level1/Level 3 pair, and the time-memory tradeoff might not be worth it. One could also try to use symmetries to perform more compression, but this again would cost a lot of bitops.)

I still haven't tested my lookup table yet, but hope to do so soon, at which point the above scheme should be implementable soon.

 **Klas Markström** · 9 June 2009

I have now put the values for k=5,6 Fujimura on my table page, and the optima solutions for small n. As you can see the number of optimal solutions grows quite fast here.
http://abel.math.umu.se/~klasm/Data/HJ/
I'm still trying to get the value for k=6 n=5.

I have found the optimal values for the weighted problem for k=5 up to n=5 and the HOC holds there. I might be able to get the value for n=6 too, but the integer program is difficult even for the commercial MIP-solver I am using now.

For k=6 the weighted problem has optimum
35/2 for n=2
140/3 for n=3

I'll try to find the optimum for n=4 later when the machine with the better MIP-solver is free.

Again it looks like the HOC fails by a larger and larger margin for larger n.

 **Terence Tao** · 10 June 2009

I fixed a bug in the code to create the lookup table (available here) and I have tested it on the sets given above. The sheer number of combinations to search, though, is daunting; even with 2222 present and using the 48 x 2 symmetries, the total number of Level1 x Level3 pairs to check is roughly 4 billion (it took me an hour just to count these pairs, and I haven't tried actually looking up their stats). And without 2222 we're looking at about 150 billion.

But it occurs to me that we can exploit the full symmetry group of the cube [3]^4, which is eight times larger than the order 48 symmetry group of [3]^3, by putting the sixteen corners of the cube (i.e. the eight corners of Level1 and the eight corners of Level3) in some canonical form before doing the pair-checking. In principle this can save a factor of 4 over our current strategy, which exploits a group of order 48×2 rather than $2^4 \times 4!$. I'll have to look at the equivalence classes of the 2^16 subsets of the corners to see exactly what one gets here...

 **Terence Tao** · 10 June 2009

OK, so I did some costing and it looks like the trick in the previous comment will reduce the number of pairs to check to 74 billion, i.e. a factor of 2 saving – not as much as I had hoped but still substantial. One new observation is that the a=1 and a=2 Pareto-optimal statistics for 4D Moser sets can be derived automatically from the a=0 Pareto-optimal ones, so they don't need to be computed separately. Indeed, given any (0,b,c,d,e) Moser set, one can create a (1,b,c,d,e) Moser set for free by adding an a-point arbitrarily. Also, a Moser set cannot saturate all of the b,c,d,e layers, so one can also get a (2,b,c,d,e) Moser set for free by adding a pair of a-points whose midpoint is in the complement of the set. So one only needs to look at the $2^{16} - \binom{16}{1} - \binom{16}{2}$ corner sets for which a=0,3,4,5,6,7,8. Thus turns out to give 396 possibilities after symmetries. For each of the 396 equivalence classes, one slices the 4D cube into 3D slices in such a way to minimise the number of Level 1/ Level 3 pairs obtained, which gives the 74 billion pairs.

There is a chance that all (0,b,c,d,e) Moser sets can in fact be completed to (3,b,c,d,e) Moser sets automatically; if so, then we can eliminate the a=3 case and this saves about 17 billion pairs to check. Perhaps someone (Kristal, perhaps?) might like to have a go at this

conjecture? (Formally: if $A \subset [3]^4$ is a Moser set with a=0, then it is always possible to add three more points to A and still have a Moser set.)

Incidentally, the a=4 case is the dominant one, occupying 27 billion pairs; for large a it seems that there aren't many Moser sets in Level 1 and Level 3, and checking the cases is relatively cheap. The a=0 case is also relatively cheap because the full 48-order symmetry group is available for Level 1, leading to about $82173^2/48 \sim 140$ million pairs to check.

 **Klas Markström** · 10 June 2009

How long does it take to check a pair with the current program? I'm wondering what the total running time is at the moment.

 **Kristal Cantwell** · 11 June 2009

I have started looking at the coloring Hales Jewett numbers. I have found some material related to this problem. One thing I have noticed is that the notation appears to vary. I have seen HJ(n) used for sets in which the lines are geometric and the in which the lines and combinatorial. I have some idea of the results for both. For geometric lines I think we can definitely improve on the results. For combinatorial lines we can obtain better numbers then there are in the existing literature but the method is known and we are just applying it to get more information about these sets. For geometric lines the method seems to be new.

 **Terence Tao** · 11 June 2009

Dear Klas,

I finally managed to debug and run a tester for Level1/Level3 pairs, and to create a second lookup table that precomputes the Pareto optimals for each choice of forbidden set modulo symmetries (and knocking out the 222 center) [the first lookup table only contained these optimals implicitly due to some additional reductions].

I was able to check 1 million Level 1/Level3 pairs in about 4 seconds, using a rather naive way to create the Level2 forbidden set (I precomputed the 125 lines through the three levels and then did a couple of bit operations for each), rather than using Michael's faster way using the a/b ancestor. So currently we're looking at about 60 hours on my (800 MHz) laptop, probably a little less if we get to use a dedicated computer. There are various further speedups of maybe a factor of 2-4 or so that I haven't implemented (e.g. exploiting residual symmetries, filling in some entries greedily, using Michael's ancestor bitmasks to compute the forbidden set, treating the 2222 case separately, etc.) but given my current error rate in coding, it may not be efficient to try to wring these final improvements in performance.

If you have about 60 hours of computing power available, I might just code up what I have without further attempts at optimisation and send it to you to run.

Best,

Terry

 **Klas Markström** · 11 June 2009

Terry,
I have quite a bit more available if there is an easy way to parallellise the problem. I also

have a fairly fast single machine which I could leave this running on over the weekend, without anything else competing for the CPU.

 **Kareem Carr** · 11 June 2009

A quick, easy way to parallelize is to take the original program, write say 6 versions of it, where in each version instead of counting through all cases, you manually code the cases you want to compute. Compile each one separately and run it on separate machines.

I don't know much about the program but if Terry is iterating through the cases in a set way, it ought to be easy to write a few versions of the program where each is programmed to compute a different and disjoint set of cases.

It's not elegant but it works and ought to require very little modification of the original program.

 **Terence Tao** · 11 June 2009

OK, I have all the C code written and (somewhat) tested. There are two programs that need to be run once (to build the lookup table), and then the third program to run the search.

The first program is at

http://michaelnielsen.org/polymath1/index.php?title=Lookup_table_C_code

(use the "edit" tab to extract a clean copy of the text.)

Running this once will create an 113MB file called "lookup.dat", which is the preliminary lookup table. On my laptop, this program took about 80 minutes to run.

The second program is at

http://michaelnielsen.org/polymath1/index.php?title=Second_lookup_table_C_code

Running this once will convert the preliminary table "lookup.dat" into a 120MB file "newlookup.dat" which is the final lookup table. On my laptop, this program took a minute or two to run. (Note that this will require about 250MB of free memory, as the program needs to hold both lookup tables in memory at once.) lookup.dat can be deleted once newlookup.dat is created.

The main program is at

http://michaelnielsen.org/polymath1/index.php?title=Scanning_code

The name of the program is not relevant, but suppose it is compiled as scan.exe. Then the format for the program is "scan x y", where $0 \leq x \leq y \leq 390$; this will scan the range [x,y] of the 391 classes of Level 1/Level 3 pairs, and dump the Pareto statistics it finds to the standard output as well as to the file output_x_y.txt. Thus, for instance, "scan 0 390" will scan all the pairs (and will probably take about 60 hours to run), and dump the statistics to output_0_390.txt. But one can parallelise easily, e.g. by having one computer run "scan 0 100", another run "scan 101 200", and a third run "scan 201 390". (All computers will need a copy of newlookup.dat, of course.)

The 391 classes have very unequal sizes (here is the list of their sizes). If one uses "scan x y count" one will see what percentage of the scanning range is covered by the range [x,y]

(and this will be very quick to compute). You can use this first to decide how to divide up the scanning tasks among your various computers.

If you want to test the code, the output of "scan 371 390" (which should take about a minute to run, but is only 0.001 percent of the total scan) can be found here and here.

Once one has all the output files for a set of ranges which covers [0,390], we can merge them, remove duplicates and non-Pareto-optimal statistics, and we will have the entire Pareto statistics for the 4D Moser sets! (assuming, of course, I have not screwed up the code.) We can of course test it against the 41, 42, and 43 point data Klas already has once it comes in, or against the linear inequalities we already know to be true.

p.s. I managed to prove my conjecture that if (0,b,c,d,e) is feasible then (3,b,c,d,e) is as well, thus allowing one to ignore all Pareto statistics with a=1, 2, or 3; see

http://michaelnielsen.org/polymath1/index.php?title=4D_Moser_brute_force_search

Because of this, the a=1,2,3 statistics are not listed in the output of the above program. "scan 0 0" will compute the a=0 Pareto statistics; this is about 3% of the entire job.

 **Michael Peake** · 11 June 2009

I don't follow the proof that (3,b,c,d,e) follows from (0,b,c,d,e). The points chosen seem to form a line 1311,2211,3111.

How easy is it to adapt the search to the xxyzw diagonals? It needs a whole new lookup table, with Pareto sets in (a,b,c,d,e,f) statistics.

 **Terence Tao** · 11 June 2009

Oops! I had had a more complicated proof, but I thought I "simplified" it into the one on the wiki which is, of course, incorrect. I'm reinstating the more complicated proof.

While in principle the search for vanilla (a,b,c,d,e) statistics can be adapted to more exotic statistics, the loss of symmetries is going to be a significant problem. Right now I am using a fair chunk of the symmetry group of $[3]^4$, which has order $4! \times 2^4 = 384$, to reduce the number of cases to check. [I'm not exploiting it fully, though: many of the 2^{16} a-signatures (i.e. the corners of the Level 1 and Level 3 slices) have some reflection or rotation symmetry, so that they lie in an equivalence class of order less than 384, and so one does not get a raw speedup of 384 from symmetry. I estimate the gain as currently exploited to be closer to 150 or so; there is room for improvement, but at this point I'll take 60 hours of computer time over 1-3 hours of my own programming and debugging time :-).] For the statistics that are relevant for the xxyzw problem I would imagine the symmetry group to be smaller by a factor of eight or so (it seems the order is now just $3! \times 2^3$ rather than $4! \times 2^4$) and the computation time would go up by a comparable factor. I guess once we know the runtimes from Klas, we'll get an idea of how feasible it would be to adapt the method, and whether we would have to come up with more clever speedup tricks.

 **Michael Peake** · 11 June 2009

Here is a proof that (3,b,c,d,e) follows from (0,b,c,d,e).

* If all b points are present, then no c points are present, so (1111),(1133),(1313) can be added.
* If any d point is present, say (2221), then we can find three absent c points (1221),(2121),

(2211), so we can add (3111),(1311),(1131)
* If e is absent, and 2333 is absent, then we can add (1111),(1333),(3333)
* If any c point is absent, say 2211, and all ds are absent, we can add (1111,3133,3311)
* If we are not done yet, then 2211,2222 and 2233 are all present.

 **Terence Tao** · 11 June 2009

Hmm, it looks like we simultaneously came up with more or less the same fix to the (3,b,c,d,e) problem. :-)

I just realised for the xxyzw problem, if one uses the x coordinate to do the Level 1, Level 2, Level 3 slicing, then one still has the 48 horizontal symmetries and vertical reflection symmetry, and the old approach (based on checking $3813884^2/(48 \times 2) \sim 150$ billion pairs) would still work (and one might even be able to reuse the lookup table). Right now we are checking about 67 billion pairs, so it's only a factor of two worse; and there are still a number of speedups available (e.g. playing with 2222, precomputing some of the forbidden set, eliminating some of the Pareto-optimals by things similar to the (3,b,c,d,e) argument, etc.). But my current code, which is focused on the a-signature approach, would take a certain amount of rewriting. (I was writing quick and dirty code that would only be used once, and so I did not employ the usual "good programming practices" that would come in handy at this stage... if one is to do more flexible searching, it's probably better to rewrite the code from scratch using a higher level language than C. I am not so familiar with Matlab, but perhaps this would be the way to go in the long run, even if C might be marginally faster.)

 **Michael Peake** · 11 June 2009

Is it possible to consider all of xxyzw, xyxzw, xyzwx, etc when building the lookup table, rather than during the main loop, and would that let you recover the symmetries during the main loop?

 **Terence Tao** · 11 June 2009

Hmm, that's an interesting idea. But this will explode the size of the lookup table, which will partially counteract the time saving. Right now the lookup tables are a manageable 110MB in size (I still find this fact hard to get used to – 110MB was *not* manageable when I first learned to program!), but this was aided by (a) the order 48 symmetry group, (b) knocking out the centre square 222, and (c) deleting some of the "easy" or "redundant" Pareto statistics such as (1,b,c,d). [The reason why there are two lookup tables in my code, rather than one, is to undo the compression from (c).] I don't know how much larger we can expect to go in memory. (My laptop is not going to like requests above 2GB, for instance, and using hard disk space rather than RAM is likely to significantly impact runtime.) But I suppose one could tolerate some expansion of the lookup table. Currently this table takes an hour to build, compared with the 60 hours needed to search pairs, so one could rebalance the time-memory tradeoff in favour of time at the expense of memory to some extent.

 **Klas Markström** · 12 June 2009

I have now done the pareto computation. I split it into 391 subcases and submitted them to one node each on a linux cluster. The 0 0 case was the hardest one and took just over 2 hours. Most of the rest took less than 5 minutes. The results are here
http://abel.math.umu.se/~klasm/Data/HJ/PARETO-DATA.tar.gz

Constructing the lookup table took 22 minutes.

Regarding the amount of memory one case use I can say that on the machines I am using each node has 16Gb of RAM.

 **Klas Markström** · 12 June 2009

That should be “the amount of memory one can use”

 **Michael Peake** · 12 June 2009

That’s great. There are 387 Pareto maxima, which include about 50 extremal points, listed at http://michaelnielsen.org/polymath1/index.php?title=4D_Moser_brute_force_search

 **Terence Tao** · 12 June 2009

Wow, that was faster than I expected (both in the run time and in getting the output collated – but parallelisation is clearly a big help; so is having collaborators evenly distributed amongst time zones). Thanks guys!

One tiny thing is that the Pareto optimal statistics were computed with the (1,b,c,d,e), (2,b,c,d,e), (3,b,c,d,e) statistics deleted (because of the lemma). The only net effect of this should be to replace the (0,16,24,0,0) maximiser by the (3,16,24,0,0) maximiser.

As a sanity check we should make sure that the statistics are consistent with Klas’s earlier computation of the 43, 42, and 41 point statistics, which can be found for instance at

http://spreadsheets.google.com/ccc?key=p5T0SktZY9DuqNcxJ171Bbw&hl=en

or on the wiki.

I see that the four 43-point statistics (5,20,18,0,0), (4,16,23,0,0), (3,16,24,0,0), (4,15,24,0,0) Klas found are among Michael’s list of extremals, and that no statistic is larger than 43 which is reassuring; we also have independent confirmation of the extremisers of the score a+5b/4+5c/3+5d/2+5e which are used in the proof of c’_5=124 (but we have a fully human proof of this fact anyway, thanks to Kristal). I put the statistics on a spreadsheet at

https://spreadsheets.google.com/ccc?key=rD2Oc_wyheVOcmCyd-Drdgw&hl=en

The next step is to find the linear inequalities and then dump them into the Maple program… we’ll see if all this effort pays off for the n=6 problem…

 **Kristal Cantwell** · 12 June 2009

Could someone post the 387 Pareto optimizers?

 **Michael Peake** · 12 June 2009

Yes, that would have made more sense than posting the fifty. They are now at the same place, http://michaelnielsen.org/polymath1/index.php?title=4D_Moser_brute_force_search

 **Terence Tao** · 12 June 2009

One consequence of the statistics is that there is a refinement of the (3,b,c,d,e) lemma: if (0,b,c,d,e) is feasible and (0,b,c,d,e) is not less than or equal to (0,16,24,0,0), then (4,b,c,d,e) is feasible. [A subtlety here: this does *not* mean that if A is a Moser set with (0,b,c,d,e) statistics, then one can add four points to A and keep it a Moser set; one may also have to rearrange

the b-points, c-points, and d-points to recover the Moser property.] It seems quite likely that this lemma could be proven by hand, thus eliminating the "scan 0 0" case and possibly shortening the run time by some significant factor. This is of course moot now, and is one of these things which was only evident *a posteriori*, but it may be something to keep in mind if one wants to try some similar search in the future.

 **Terence Tao** · 12 June 2009

Grr... there is a bug in the code somewhere; (8,32,0,0,0) ought to be Pareto-optimal, but is not present in the lists. I'll try to track down the source of the problem...

 **Terence Tao** · 12 June 2009

OK, I found the problem: when I manually added in the (a,0,0,0) Pareto optimals in the second lookup table from the first, I made the mistake of thinking there were 4 a-points in [3]^3, when of course there are 8. This should only impact a small fraction of the statistics (only the ones where the Level2 statistics are of the form (a,0,0,0), which is for instance the case with the (8,32,0,0,0) example), but unfortunately I think have to rerun the whole code.

Klas, I had to change the code to create the second lookup table, at

http://michaelnielsen.org/polymath1/index.php?title=Second_lookup_table_C_code

and also the scanning code at

http://michaelnielsen.org/polymath1/index.php?title=Scanning_code

(because I had to increase the number of Pareto stats per forbidden set by one, from 26 to 27). The first lookup table, lookup.dat, does not need to be recomputed (hopefully you didn't delete it like I said you could), but the second lookup table, newlookup.dat, does need to be, and will be marginally larger now (something like 123 MB instead of 120 MB). Unfortunately I think the entire range 0-390 needs to be rescanned.

 **Kristal Cantwell** · 12 June 2009

Thank you for posting the Pareto optimizers. From the statistics it looks like a Moser set with its center point can only have 36 points. I think that was conjectured earlier of course there could be more such Pareto optimizers in the range 0-390 that may need to be rescanned.

 **Kristal Cantwell** · 12 June 2009

I think if all Moser sets with n=4 that have their center points have less than 36 points then all Moser sets with n=6 that have their center point must have 360 points or less. To see this for any such set slice along any two coordinates the slice 22 will have 36 points and the remaining 648 points will be symmetric about the center point and hence at most 324 of them will be in the Moser set for a total of 360.

 **Klas Markström** · 13 June 2009

I had kept the file, I usually try to keep things which took some time to construct for a while in case they are needed again, so it did not take long to rerun the program. The new files are here
http://abel.math.umu.se/~klasm/Data/HJ/PARETO-DATA-NEW.tar.gz

By the way, my c-compiler, gcc 4.1.2, complains about there being a function named “index” so in order to compile the programs I just changed that name to “index1”

 **Michael Peake** · 13 June 2009

There were 390 pareto maxima this time, including (8,32,0,0,0). They are now at http://michaelnielsen.org/polymath1/index.php?title=4D_Moser_brute_force_search.

 **Terence Tao** · 13 June 2009

I computed the extremals from Michael’s data. I got 154 extremals, far more than the 50 in the previous run… so I am a little worried as to the integrity of my computations, it would be good to have a double check. (My list of extremals is at http://michaelnielsen.org/polymath1/index.php?title=4D_Moser_brute_force_search).

I have code now to extract out the linear inequalities; there seem to be a lot of them, but it should be computable.

 **Terence Tao** · 13 June 2009

Ah, I see what I did wrong – I only computed those points which were not convex combinations of two other points, when of course I should be looking at combinations of up to five points. Never mind…

 **Michael Peake** · 13 June 2009

There might be another problem: You kept (5,16,19,1,0), but the original list includes (5,15,20,1,0) and (5,17,18,1,0)

My original list of fifty was found by the doubtful method of
* Multiply the 390×5 matrix by a random positive vector
* The row with the highest value is an extremal.
* Repeat ten million times.

I’m not sure how else to do it. This time, I have just removed those points that are midpoints of two others, and got down to 113 points. Then I form a hyperplane from any set of five that form an invertible matrix, and check whether it is extremal. It is up to 106 linear combinations, and more than two-thirds finished.

 **Terence Tao** · 13 June 2009

I found a great package, qhull, from the geometry centre,

http://www.qhull.org/

which computed the extremals for me; there turn out to be 58 (now on the wiki). I also brute forced the associated linear inequalities (by checking the $\binom{58}{5}$ combinations), now also on the wiki (in a random order, and possibly not all in least common denominator yet; there are currently 140 inequalities there.

Hmm, I see that you just did the same computation, and got 145 inequalities instead. Hmm, we’ll have to see exactly how they differ…

 **Terence Tao** · 13 June 2009

OK, it looks like my inequalities are a strict subset of yours; I'm not sure how my program managed to miss some of them, but it looks like all of yours are correct, at any rate.

Time to fire up maple...

 **Terence Tao** · 13 June 2009

OK, I dumped the 145 inequalities into the Maple routines at

http://michaelnielsen.org/polymath1/index.php?title=Maple_calculations

and (after clearing out some errors in those routines – sigh) it looks like the upper bound for c_6' has improved from 361 to 356, coming within range of Kareem's 353-point solution!

What I am going to do next is make sure Kareem's 353-point statistics are actually feasible for the inequalities we have, as a sanity check...

 **Terence Tao** · 13 June 2009

Yes, the statistics (22, 66, 165, 100, 0, 0 0) of Kareem's example is indeed feasible. Reassuring...

Maple also tells me that if g=1 then a 6D Moser set can have at most 337 points – so we can assume g=0. If f=1, then a 6D set can have at most 353 points, so we can also (barely) assume f=0.

Unfortunately, for e>0, one can have as many as 355 points, and for 354 sets, e can be as large as 5, so we can't quite reduce to e=0 yet.

On the other hand, Maple also tells me that there is only one statistic possible for 356-point Moser sets in [3]^6, namely (24,72,180,80,0,0,0). So there is a good chance one can reduce 356 to 355, at least, thus narrowing the range for c_6' to [353,355].

The range for c_7' is still rather poor: [988,1041].

 **Terence Tao** · 22 May 2009

I propose, by the way, that the paper be written in LaTeX, as it is a standard format for mathematical papers and also supports everything we need, such as tables and pictures. We have some interesting visualisations of various subsets of [3]^n, for instance, which it would be good to have in the paper at some point. (But the specific graphics to put in will probably only be decided upon quite late in this process, once most of the text is already in place.)

 **Kristal Cantwell** · 23 May 2009

The outline looks good, thank you for posting it. In terms of c_5′ the one computer result that is used in the proof that looks hard to get rid of is when the size of a four dimensional Moser set is 41 and the number of points with two twos is less than 17 then there must be exactly 12 points with two twos. I don't see why that is forced. It is an interesting problem.

 **Terence Tao** · 23 May 2009

Hmm. That does sound like a difficult fact to prove by hand. Perhaps it may be more efficient to try to reduce the dependence of this fact in the n=5 proof. On the wiki it seems that we only use this fact in the proof of Lemma 2.

One possibility here is to reprove Lemma 2 by doing it in stages. Instead of jumping right away to the proof that the middle slice has at most 41 points, first prove that it has at most 42. I think we actually established this at some earlier stage of the project, and this should be doable without relying too much on the computer data. (For instance, the 3D data and maple tell me that 43-point sets must have c at least 18, so the Lemma 2 argument already seems to work here.) Once we have this, we know that the *2*** slice has at most 42 points, so for a 125 point set at least one of the *1*** and *3*** slices have at least 42 points, and thus have d at most 2. Similarly for the **1** and **3** slices, etc. This suggests that the d-points of the 124 point set have to either be very few in number, or very irregularly distributed. Hopefully this can be used somehow (without having to use the fact that c(2****) is either at least 18, or is of a very special form).

At the worst case, we can try to minimise and isolate the few computer facts that we absolutely need, and leave it as a challenge for future authors (or future polymath1 participants) to find human proofs of those facts (or at least proofs that can be easily and transparently computer-verified by standard software, e.g. Maple).

 **Terence Tao** · 23 May 2009

OK, I played around with maple and got the following new fact:

– 40-point Moser sets in [3]^4 have a d-statistic of at most 6.

(Details are in http://michaelnielsen.org/polymath1/index.php?title=Maple_calculations .)

We already know that 43-point sets have $d = 0$, 42-point sets have $d \leq 2$, and 41-point sets have $d \leq 3$ (and these facts are mostly computer-free at this point.) As a consequence, given a 125-point Moser set, if we know that middle slices have size at most 42 (so that side slice pairs must be at least 40+43 or 41+42), we conclude that

$d(1****) + d(3****) \leq 6$ (*)

and similarly for permutations; if a middle slice has size at most 41, then the 6 improves to a 4. Double-counting (as in Corollary 4 on the wiki) we see that every middle slice has a c-value of at most 12. On the other hand, the only way 12 can actually be attained is if every middle slice has exactly 42 points and (*) is obeyed with equality; Maple tells me that 42-point sets have c at least 12, so c(2****)=12 and similarly for permutations. But double-counting this fact, we see that the average value of $d(1****) + d(3****)$ must now be 8, contradicting (*). So in fact middle slices have a c-value of at most 11 and so must have at most 41 points, giving Lemma 1.

 **Terence Tao** · 23 May 2009

A trivial issue to settle here: should the paper use American English or Commonwealth English? The rule I usually use is that I write in Commonwealth English if a strict majority of the authors are from the Commonwealth, and American otherwise. I know Michael is an Aussie like myself, and Christian is German but works in the UK, so it seems to be more or less a tie here… of course, it's easy to change from one to the other, so this is not a major issue in any event.

 **Terence Tao** · 24 May 2009

I've written up a draft of the Moser portion of the paper now, the PDF is at

Click to access polymath.pdf

and the source files are at

http://michaelnielsen.org/polymath1/index.php?title=Polymath.tex
http://michaelnielsen.org/polymath1/index.php?title=Moser.tex

It's actually rather lengthy to put everything together (27 pages!). Also, one is still missing the human proofs of the fact that d <= 3 for 41-point Moser sets in [3]^4 (one also needs d<=2 for 42 point sets and d=0 for 43 point sets), as well as human proofs of the inequalities for [3]^3 that Michael found. But other than that, the proof of $c'_{5,3} = 124$ is now computer-free, albeit tediously long.

I didn't quite realise just how large the paper is going to be – once one puts everything else in as well, it's looking like 50+ pages. We may eventually want to abridge some of it, referring to the wiki for further details.

Feel free to make minor changes to the text so far by editing the source directly; I plan to make the wiki files the master version of the source (i.e. I will download from the wiki whenever I want to recompile the PDF or make other changes). If you want to download the source and make major changes, one should probably notify the rest of us here to avoid bifurcation of the version tree.

In a few days I plan to start porting the c_n bounds from the wiki to the LaTeX source.

This would be a good time to make any comments on notation, exposition, etc.; it will be easier to make these sorts of changes now, when only a portion of the paper is written, than later, when there would be a lot more text to re-edit.

 **Kristal Cantwell** · 24 May 2009

That seems to take care of lemma 2. I think the remaining parts of the proof should be easier to clear of computer generated results. I don't have any problem with Commonwealth English for the paper but then I don't really have any experience writing it. I would think any problems involved in standardizing the paper this way would be minor.

 **Kristal Cantwell** · 24 May 2009

I think the proof of d <= 3 for 41-point Moser sets in [3]^4 is at the end of section n=4 in the wiki at
http://michaelnielsen.org/polymath1/index.php?title=Moser%27s_cube_problem

It is looking good so far I think.

 **Kareem Carr** · 24 May 2009

I took a look at the outline Looks good. I was wondering if my input was needed in any way for the parts concerning the genetic algorithm.

 **Terence Tao** · 24 May 2009

Dear Kareem,

I would imagine that the genetic algorithm would be discussed in the section on the lower bounds for Moser, which hasn't really been written yet. If you like, you could write a section on it right now (describing the algorithm, and mentioning the lower bounds obtained) and we'll insert it somehow into the draft. (We should also have a discussion on the integer programming techniques, even though we have been making an effort to become independent of them now.)

Dear Kristal: thanks for the pointer. I'll probably try to port that argument over at some point in the near future.

 **Kareem Carr** · 25 May 2009

Dear Terry,

I started writing something up. It should be done by the end of today (east coast time).

 **Student** · 25 May 2009

Hi,

May I recommend using something like Dropbox https://www.getdropbox.com/ for sharing and working on the LaTeX source tree. This way if you share the source directory with your collaborators they can have a look at the changes as they get synced and also add to it.

It will probably not solve merge conflicts it two or more people are working on the same file, but perhaps if each collaborator works on his own file it might be an easier solution than using cvs/svn/git etc.

The great thing about Dropbox is that it is available under Windows, Mac and Linux, and is quite easy to use as its just another folder.

 **Terence Tao** · 25 May 2009

I have incorporated Kareem's file (which was sent by email) into the newest version of the paper at

Click to access polymath1.pdf

The appendix on genetic algorithms source file is at

http://michaelnielsen.org/polymath1/index.php?title=Genetic.tex

Kareem (or anyone else): please feel free to edit the source file; it is the master copy and I will use it (together with the other source files on the wiki) to generate new PDF versions whenever there are major changes to the source.

 Ryan O'Donnell · 26 May 2009

I too wondered about American vs. Commonwealth English for the other paper. Between me, Tim, Terry, Jozsef (works in Canada)… there might be enough Commonwealth people to tip the scales to "colour" and "analyse". On the other hand, this might get settled by the editorial policy of the journals to which we submit.

I also wonder whether the two papers might use different initials for this Polymath author, to reflect the fact that the papers were developed via two essentially separate projects (albeit sharing a common wiki and some participants).

 **Michael Peake** · 27 May 2009

I translated most of the wiki page on low c_n values into Latex, and included it in polymath1.tex. I don't have a Latex compiler, though, so I am sure to have left out a bracket : (.

 **Terence Tao** · 27 May 2009

Thanks Michael! I formatted the text and split up the files a bit. The latest compiled version can be found at

Click to access polymath2.pdf

It's 44 pages right now, although this includes a few pages of whitespace which I will try to remove later at some point.

There is of course a lot of work to be done in harmonising the notation, cleaning up the non-rigorous language from the wiki into a more professional style, adding pictures etc., but this is a good starting point at least, and will give a rough idea as to what we are evolving towards.

I also started putting in stubs for the other source files (fujimura, higher k, etc.) at

http://michaelnielsen.org/polymath1/index.php?title=Outline_of_second_paper

but there is not much there yet.

Constructive criticism and suggestions for the paper are, of course, very welcome; there is still a fair way to go before this paper reaches a professional level of quality.

 **Kristal Cantwell** · 28 May 2009

I have a proof for d=0 for Moser sets with 43 points. I will post it in several parts:

If a Moser set with n=4 has 43 points it has no points of type d.
Proof: Assume that such a set exists. We have already proved that it contains no point of type e. So we can assume that. This set does not contain such a point.

This set since it has 43 points must have at least 18 points of type c. See http://michaelnielsen.org/polymath1/index.php?title=Maple_calculations.
Since there is a point of type d we can slice the set so that one side cube has its center slot filled and so must have 13 points or less.
The other cubes must sum to 30. The other side cube must not have 16 points since then by the list of Pareto optimizers its distribution would be (4, 12, 0, 0) and it would contribute no points of type c to the Moser set. Then the other side cube could contribute at most 3 points of type c. So for there to be 18 the central cube must contribute 15 which is not possible.

This means the center cube must have at least 15 points for there to be a total of 43. Now we will show that the configuration has at most one point of type d.

Assume that there are more than one. Then no two can have the same c-statistic or we can slice and get two side cubes with center points filled which give each a total of at most 13 points which would force the center slice to have 17 points which is impossible.

So we can get a slice that has one point of type d in a side cube and the other in the center cube. That means the side cube must have its center slot filled and it must have 13 points, the center cube contains one point of type c(type d in the overall Moser set) and thus must have 15 points or less as the only Pareto optimal set with 16 points contains none of type c. The remaining side cube must have 15 points or more.

The only distribution which works is 13 points in one side cube and 15 in the other two. Now since there is no Moser three set with one point of type c and a total of fifteen points. There must be at least two points of type c in the center cube in giving at least 3 for the overall set. From this and the Pareto optimizers we see that the distribution of the side slice with 13 points must be (3, 6,3,1) or (4,6,2,1) and of the center and side cubes (3,9,3,0) or (4,9,2,1) This means we have at least 17 points of type b. Every one of these points appears in the center cube in exactly one slice. There are 4 slices so there is one slice with 5 of these points in the center cube.

end of part one, part two follows.



Kristal Cantwell · 28 May 2009

part two of proof every Moser set with 43 points has no point with three 2's:

Now since we have at least three points of type d and we can't have two of them in the side cubes we must have at least one in the center cube. This means that that cube must have at most 13 points. No if either of the side cubes has a point of type d its total will be at most 13 and the third cube must be 17 which gives a contradiction so all three center points must be in the center cube.

This lowers the total of the center cube to at most 12. This in fact forces the center cube to have distribution (5,4,3,0) or that distribution with one point removed.. This also forces the sides cubes to have total 31. This means that one will have 16 points and thus have statistics (4,12,0,0). The other will have 15 points and hence have 9 point s of type b giving a total of 25 of type b if the center cube has 12 points.

If it has 11 points then it will have the distribution (5, 3, 3 as that is the only subset of the Pareto optimizers satisfying the conditions. Then both side cubes will have 16 points which means that neither will contain points of type c which means that there must be 18 points of type c in the center cube which gives a contradiction.

So the center cube has 12 points and the total number of points of type b is 25 so since each point of type b occurs in one slice and there are four possible slices there must be one slice with 6 points of type b since we get a contradiction as above if there are less than 11 points. So there must be 12 points the only distribution that works is (6,6,0,0).

But then the middle slice contains none of the points of type d and one must be on a slide slice lowering the total of that slice to 13 or less forcing the other side slice to be above 18. So we have reached a contradiction in assuming that we have more than one point of type d the remaining case is that there is exactly one point of type d.

If there is exactly one slice so it is in one of the side cubes. That cube will have at most 13 points. The other side cube can have at most 15 points because if it has 16 it will have no

points of type c and the other side cube will have at most 3 of type c and since the total size is 18 we must have 15 of type c in the center cube which gives a contradiction.

So the center cube must have at least 15 points. Furthermore since it has no points of type c since we have our single point of type d in one of the side cubes it must be a subset of (4,12,0,0) and must have at least 11 points of type b. To avoid forming a monochromatic line the points of type b of the side cubes must sum to 13 or less.

End of part, part three follows.



Kristal Cantwell · 28 May 2009

Third and final part of proof that every Moser set with 43 points contains no point with three 2's:

If the side cube without the center cube has 15 or more points it must have at least 9 points of type b forcing at most 4 of type be in the other side cube which forces its size to 11 or less which forces both of the other cubes to have more than 16 which contradicts the fact that no side cube can have more than 15 points proved above.
So the side cube without the center point must have at most 14 points. The other side cube can have at most 13 points so the center cube must have 16 points and since it cannot have 17 we have the distribution (13,16,14).

This forces the center cube to have all its points of type b there are 12 of these. From the Pareto optimizers the side cube with 13 points with the center spot filled must have at exactly 6 points of type b. We will get a line unless the remaining side cube has more than six points of type b. The only distributions with at least 14 points that satisfies this is (3,6,5,0) and (2,6,6,0). From the Pareto optimal statistics we can limit the possibilities for the other cubes of slice and get the following:

So one center cube has statistics (4,12,0,0) the side cube with thirteen points and the center filled has distribution (3,6,3,1) or (4,6,2,1) and final cube has statistics (3,6,5,0) and (2,6,6,0). So the statistics of the Moser set must be (6,16, 21,1,0), (7,16, 20,1,0), or (8,16, 19,1,0). So the exact number of points of type b must be 16.

Now slice the set so that the point of type d is in the center slice. Then the center cube contains a point of type c and has at most 10 points of type b. It must have at most 14 points that means that the other two cubes must total at least 29. If one is 16 then it contains no points of type c and since the center cube contributes at most 10 the other side cube must have 8 such points. This gives a contradiction which means one side cube must be of size 15 and the other of size 14. The one of size 15 contains at most three of type c. The center contains at most 10 so the remaining cube must have 5 or more.

Again from the Pareto statistics we can limit the statistics of each of the cubes the side slice of size 15 must be (3,9,3,0) (4,9,2,0) otherwise it will have no points of type c and we will get a contradiction as above. The side slice must have at least 5 points of type c and at least 14 points so it must have statistics (2,6,6,0) or (3,6,5,0). The center slice must have 1 point of type c in its statistics and at least 9 points of type b in its statistics which are statistics of type in the whole Moser set. Thus it statistics are either (4,9,1,0) which is a Pareto optimal set with one point removed or (3,10,1,0). Then the distributions are either (5, 19,18,1), (5,18,19,1), (6,19,17,1,0),(6,18,18,1,0), (7,19,16,1,0) or (7,18,17,1,0). Since we must have 18 points of type c

the only possible distributions are (5, 19,18,1), (5,18,19,1) or (5,18,19,1). But we have shown the exact number of points of type b must be 16 which is a contradiction and we are done. So every Moser set with d=4 cannot contain a point with three two's.

 **Kristal Cantwell** · 30 May 2009

I have added the proof that every 43 point has no point with three 2's to the wiki at

http://michaelnielsen.org/polymath1/index.php?title=Moser%27s_cube_problem

It is in the section n=4 following the result I put in the same place earlier that every 41 point set has three or less points with three 2's.

 **Kristal Cantwell** · 31 May 2009

I finished the proof that a 42 point Moser set must have 2 or less points with three 2's.
I posted it in the wiki at http://michaelnielsen.org/polymath1/index.php?title=Moser%27s_cube_problem at the end of the section n=4. I also posted it here but since it was so long It did not go through.

 **Terence Tao** · 31 May 2009

Thanks Kristal! I am going through the paper now to write an introduction and reformat some of the existing text, and will incorporate your new material shortly.

Incidentally, Ryan (whose comment was initially trapped in the spam filter, sorry!) raised the question of how to attribute authorship here. I like the idea of having just the Polymath pseudonym as author (but we can debate what the first initial should be :-). We may also want some section listing contributors to the project, though there is a delicate issue of where to draw the line between "major" contributors and "minor" contributors – does a single comment on a thread count as a contributor? I can see the case of not trying to list contributors at all to avoid this problem. (I may need eventually to list a corresponding author, though, to handle journal submission.)

 **Klas Markström** · 31 May 2009

I am now back home and beginning to be free of jet lag as well.

I can start out by writing a section describing the integer programing methods I have used for those of the computational parts I have done.

 **Terence Tao** · 31 May 2009

Dear Klas:

Sure, please do! The relevant file for this is

http://michaelnielsen.org/polymath1/index.php?title=Integer.tex

I've written an introduction, added in Kristal's Moser stuff (thus making the Moser bounds up to n=5 computer free except for Michael's Pareto-optimality calculations for n=3), at

Click to access polymath3.pdf

Michael, was there anything special you did to obtain the n=3 calculations? I guess the 2^{27} ~ 10^8 combinations can simply be brute-forced in a matter of minutes on a modern computer, so nothing fancy is needed.

 **Michael Peake** · 31 May 2009

There are shortcuts to the 2^{27}. For example,

* There are 230 line-free subsets of the square.
* Choose a first layer, choose a third layer.
* Find which second-layer cells do not complete a vertical or sloping line.
* Identify the list of pareto-optimal second layers that are made from those cells. There are 512 of these lists, average length 3, longest length 23.
* Build the cube, find its statistics, compare with the current list of pareto-optimal statistics.

The first point saves a factor of 11, the second saves a factor of 2, and the fourth saves a factor of between 10 and 100.

 **Kristal Cantwell** · 1 June 2009

I am going to start showing that each of the Pareto optimal configurations are Pareto optimal by hand. I will put it at the end of the n=3 section of the Moser wiki and periodically post what I have done so far here.

 **Terence Tao** · 1 June 2009

Dear Kristal: Thanks for this. It may help to first compute the Pareto-minimal counterexamples (i.e. the (a,b,c,d) which are larger than those in Michael's list, but end up in Michael's list after removing one from any of a,b,c,d) and to eliminate each of them. From the n=2 inequalities (and using the planes xy1, xy2, and xxx, averaged over symmetries) we have a number of linear constraints

$$a/4 + c/6 \leq 2; a/4 + d \leq 2; b/6 + c/6 \leq 2; b/6 + d \leq 2; c/3 + d \leq 2; a/2 + b/6 + c/6 \leq 4; b/3 + c/3 + d \leq 4$$

which should be helpful in eliminating many of these; these inequalities, by the way, together with the trivial inequality $d \leq 1$, are given as Y3 in the maple computations on the Wiki.

One should also be able to get one or two more inequalities by using the skew planes xxy as per Michael's method, but I haven't computed what they are. Anyway, it may be a good idea to do this first, before doing the labor-intensive program of eliminating each of the possible counterexamples by hand.

 **Terence Tao** · 1 June 2009

Hmm, the technique wasn't quite as effective as I had first thought. It seems that the Pareto-minimal counterexamples that are consistent with these inequalities are (4,5,3,1), (3,11,1,0), (4,10,1,0), (5,8,1,0), (6,5,1,0), (5,7,2,0), (6,4,2,0), (4,8,3,0), (5,5,3,0), (6,0,3,0), (3,8,4,0), (4,7,4,0), (5,0,4,0), (3,7,5,0), (4,5,5,0), (3,6,6,0), so one has to eliminate 16 possibilities – not too pleasant, unless one can find some way to do them in batches. Also, it looks like the xxy planes don't give any new inequalities, so no luck there.

 **Jason Dyer** · 1 June 2009

I've given the paper a thorough look ... this really was quite a substantial effort!

I do think we should be more careful about stating the "open problems" left remaining (for example, in the saturation section it states (4,4) and (4,6) are not saturated ... and then the text cuts off).

I have time Friday and will try writing the Fujimura section then.

I can also try the geometric / combinatorial picture.

 **Jason Dyer** · 1 June 2009

Regarding the picture ...

Dimetric? Isometric? Does anyone care?

Numbers on or off? (Because this is black and white I'm inclined to leave them off, it's too hard to handle the visibility with the lines crossing over.)

Showing [3]^3 side by side ... should it be combinatorial on the left and geometric on the right, or vice versa?

 **Terence Tao** · 1 June 2009

Dear Jason: Thanks for helping out! I think whatever picture looks best is fine; we can always use the caption to explain what the interpretation of the picture is.

Dear Michael: These are neat speedup tricks for the brute force search. I thought of another one which could conceivably save another factor of up to 8. You said that there were 230 line-free subsets of the square; but many of them are going to be equivalent, using the symmetry group D_4 of the square which has order $2^2 \times 2! = 8$. So one can group 230 into about 30 or so equivalence class, each of which typically has size 8 (there are a few that are smaller, but generically one expects each equivalence class to have the full size of 8).

We can arbitrarily order the ~30 equivalence classes (thus giving each of them a number from 1 to ~30), and then arbitrarily order the representatives within each class (giving them a number from 1 to 8).

Now, by symmetry, when considering the first and third slices, one can assume that the equivalence class of the first slice has a equal or lesser number than that of the second slice (saving the same factor of 2 that you saved), _and_ one can assume that the first slice is the _first_ representative of the equivalence class that it is in (thus saving an additional factor of up to 8).

This brings up the possibility that one might now be able to compute the Pareto-optimal table for $[3]^4$. A pure brute force would take 2^{81} computations, which is infeasible. But with all these speedups, I think there is a chance. Firstly, the total number of line-free sets in $[3]^3$ is at most $230^3 \sim 10^7$; actually, the number is likely to be significantly less than this, let's say of the order of 10^6. It should not be hard to enumerate all of them. Then, we have the symmetry group of $[3]^3$, which has order $2^3 \times 3! = 48$. Dividing this out, we are looking at something like 20,000 equivalence classes of line-free sets, each with up to 48 representatives. So the outer loop here is going to be over about $\frac{1}{2} \times 20,000 \times 20,000 \times 48 \sim 10^{10}$ combinations; a bit large, but within the range of modern computers.

But what happens inside the loop? We have to prepare a lookup table for each of the 2^{27} possible sets cut out by non-horizontal lines between the first and third layers. Each element of the lookup table consists of a list of Pareto-optimal quadruples. Given that there are 22 Pareto-optimal quadruples for the entire set $[3]^3$, it is reasonable to expect that the average list will have length ~ 20, leading to a lookup table of about 2GB in size, which is quite manageable. It looks like processing the loop for each choice of first and third slices would take about 10^5 bit operations or so (being conservative), for a total computation cost of maybe 10^{15} bit ops, which should be performable in a matter of weeks on a typical computer (less, if one programs efficiently).

The one tricky thing would be to build the lookup table. Given $\sim 10^6$ line-free sets in $[3]^3$, a brute force approach would require matching $2^{27} \times 10^6 \sim 10^{14}$ pairs, which assuming 10^5 bit ops to compare each pair would take centuries. But one can use the symmetry group to cut down the 2^{27} combinations by a factor of about 48. One might also be able to cut down the amount of computation by grouping the $\sim 10^6$ line-free sets according to their (a,b,c,d) statistics; once a hit is found for one line-free set in its (a,b,c,d) class, none of the other sets in that class need to be checked, and none of the sets in classes strictly smaller than that class need to be checked either for the purposes of Pareto optimality. This could conceivably shave a factor of, say, 10 from the running time. This brings the total computation required to the level of months; it is easily parallelisable, so given several computers one could do it in a reasonable period of time.

I'm not sure this is really worth the effort yet, though; it would give a whole bunch of inequalities which we could use to improve the upper bound on $c'_{6,3}$, but it's still unlikely that we will come that close to the lower bound coming from the GA (the current range is [353,361]).

One thing which might be computable pretty quickly though is the exact number of line-free sets in $[3]^3$, and how many equivalence classes there are under symmetries. This would give us a more accurate idea of whether the above computation is really feasible.



Terence Tao · 1 June 2009

Actually, my initial computations were bit too conservative. To compare whether a centre slice fits inside one of the 2^{27} subsets of $[3]^3$, one merely needs to AND together two 27-bit (i.e. 4-byte) strings together and check whether the result is zero or not, which should take considerably less than 10^5 bit operations.

Another thing to do is to separate into two cases, one where the set contains the centre 2222 and one where it doesn't. When it doesn't, one only has 2^{26} sets to consider rather than 2^{27}, and the number of line-free sets in the slice should also go down slightly (and the management of statistics should be slightly easier too). On the other hand, when one does have 2222, then there should be far fewer line-free sets to deal with, and also the first and third slices are much more constrained, so the loop should run a lot faster. This may end up saving another factor of 2 or so... not too much, but these savings add up after a while.



Kristal Cantwell · 2 June 2009

Thank you for the elimination to 16 cases. The problem as it is looks doable to me.

 **Kareem Carr** · 2 June 2009

Regarding the pictures, are the sorts of pictures that I have on my website useful? Because I have code that can automatically generate these for arbitary elements in [3]^n from a given list of elements.

Interesting discussion of speedup tricks. I suspect it might be relevant to the GA. I would not be surprised if my current representation of solutions can be drastically improved.

Concerning the GA, I am thinking of organizing a group computation if people would be interested. I could distribute a pair of 32-bit executables that would run on Windows and Linux computers. If a large number of computers could be used, we might increase the probability of better results. Is this something that people would be interested in?

 **Michael Peake** · 2 June 2009

There are 3813884 line-free subsets of [3]^3. These form 83158 equivalence classes: 76066*48+6527*24+51*16+338*12 +109*8+41*6+13*4 +5*3+3*2+5*1

In the case without the 2222 point, the centre of the third layer does not affect the second slice, and the six points next to the centre only affect one cell each of the second slice. So it should be quick to calculate which cells the second slice is constrained to.

 **Terence Tao** · 3 June 2009

Dear Kareem: I think the pictures you have will indeed be useful for visualising higher-D sets. Kevin O'Bryant kindly contributed a sample 2D picture which is now on the new version of the paper at

Click to access polymath.pdf

but for 3D and higher I think your software is the way to go. One could for instance have a nice picture depicting maximal line-free and Moser sets in low dimensions, which we could then put in the "lower bound" sections of the paper…

Incidentally I was playing around with the [3]^6 Moser set examples. Kareem's 353-point example does seem to be significantly better than the type of sphere-packing examples we had before; I could only get up to 342 points that way, though perhaps I wasn't being maximally efficient. There does seem to be an interesting phase transition at six dimensions, where the maximisers become much more chaotic.

 **Terence Tao** · 3 June 2009

Dear Michael: Thanks for the computations! They came in a bit larger than what I was hoping, but still not too bad. After using the symmetries, the number of first slice/third slice pairs that one has to deal with is roughly $\frac{1}{2}76066^2 \times 48 \sim 1.4 \times 10^{11}$ – large but not impossibly so.

In the case when 2222 is omitted, one can greedily fill in the 1222 and 3222 spots, which should cut down the 1.4×10^{11} possibilities for the first and third slices a little bit. But the lion's share of the work here is still in crafting the lookup table; there are 2^{26} possible configurations for the forbidden portion of the second slice cut out by the lines connecting the first and third slices (but after dividing out by the symmetry group this should have size

about $2^{26}/48 \sim 1.3 \times 10^6$), to be compared against the 3813884 line-free sets (actually we only need those sets which omit the centre point 2222, which should cut the latter down a little bit). That's close to being brute-forceable as is (we're looking at about 5×10^{12} comparisons), but one should be able to speed this up a bit by first sorting the 3813884 sets into (a,b,c,d) classes, working with the large ones first, and skipping the rest of a class (and all classes below it) as soon as one gets a "hit" in that class.

With the centre point 2222, things look pretty good, in large part because I believe the number of line-free sets in the second slice should be cut down substantially (for instance, just by considering the 13 rays through the center of $[3]^3$ I get a crude bound of $3^{13} \sim 1.4 \times 10^6$, and the truth should be even lower than this). And most of the 1.4×10^{11} first-slice/third-slice pairs should be eliminated instantly due to the 2222. So it's beginning to look feasible...



Terence Tao · 3 June 2009

Kareem, it occurred to me that 230 fits nicely inside a byte range [0,255]. So one could store a Moser set in $[3]^6$, for instance, as a string of $3^4 = 81$ bits. (One might want to pad out the list of 230 with 26 additional line-free sets to make a clean 256, e.g. by taking 26 line-free sets from the best examples of Moser sets we already have.) It should not be hard to build a lookup table of 256^3 bits (= 512 MB) declaring which triples of 2D line-free sets give 3D line-free sets (Michael may already have done this, in fact; also, one can use symmetries to reduce the size of the table). One could then test whether a set in $[3]^6$ is line-free by checking this lookup table once for every line in $[3]^4$. The number of such lines is

$$1 \times 40 + 8 \times 13 + 24 \times 4 + 32 \times 1 + 16 \times 0 = 272,$$

so by storing those linesets in a lookup table too one should be able to test for line-free-ness in a quick manner. (The downside, though, is that if there are lines, it is difficult to decide how to modify the set to eliminate the lines. I guess one could augment the 512 MB table by associating to each non-line-free set in $[3]^3$, a line-free set that can be obtained by eliminating as few points as one can from the set (this may explode the size of the lookup table, though, unless one uses symmetries). Applying this procedure once for every offending line, one should get back to a line-free set.)

Coming back to the statistics for $[3]^4$ sets, it occurs to me that one can split up the computation in a way that should speed it up substantially. For instance, define the *signature of a set* in $[3]^3$ to be its intersection with the set {222, 221, 223, 212, 232, 122, 322}. There are $2^7 = 128$ possible signatures (and significantly less, if one uses symmetry), so one can subdivide the 3813884 line-free sets into 128 classes based on signature (this is going to be a rather uneven distribution, though). So, for each of the 2^{27} subsets of $[3]^3$, one does not need to compare against all 3813884 sets, but only those whose signature is disjoint from the signature of the set being considered, which should cut down the number of comparisons significantly, especially when combined with the symmetry reduction.



Jason Dyer · 3 June 2009

Rereading the paper: we found a counterexample to the HOC for k=4 but I don't believe we addressed larger k. Unless there's a trivial argument involving subsets that I'm missing to dispense with all larger k, we should at least test k=5 (in case primality is a factor).

 **Kevin O'Bryant** · 3 June 2009

I just put in a rewritten proof of Theorem 1.3 (what was there was incorrect). We can run pretty much the same proof to give a construction for [k]^n. Maybe this should just replace Theorem 1.3, or maybe it's wise to keep the simpler version here, and put the general version in the "higher k" section.

I haven't worked out the theorem for [k]^n, just the proof.

 **Kristal Cantwell** · 3 June 2009

I started working on the Pareto minimal conterexamples. I got through the first two. What I have is at the end of section n=3 on the Moser wiki.

 **Kevin O'Bryant** · 4 June 2009

The asymptotic lower bound really uses a subset B of [n]^2 that is free from 3-term APs. One way to construct such a thing is take a subset A of [n] that is 3-free, and then use B=A x A. I'd be surprised if there was a better construction known, but am I wrong to suspect that there are larger B that are not simple products?

 **Kristal Cantwell** · 4 June 2009

There is a counterexample to the HOC for all even n greater than 2. One takes one element on the diagonal say (n,n) then one can take (a,a+1) for all n-1 vaules 0 through n-2 and (n-1,0) this will block all combinatorial lines and it has weight (n+1)/2. The complement of this thus line free. Its weight is greater than any set of slices. For each such set must contain at least one diagonal blocking point and n-1 other blocking points if it has two or more diagonal blocking points the complement will be less than the above set(there must be at least n blocking points so 2 or more diagopnal points increase the weight by 1/2 and pushes it over the limit constructed above. If it has one then because of parity reasons it must have an additional point (there are an odd number of coordinates to take care of and since if (a,b) is in the blocking set (b,a) must be as well, coordinates must be blocked by pairs there is one left over which requires an addition point or an addition point of the form (a,a) which makes the line free set lower than the above construction. The HOC is known to be true for 2 and for even numbers greater than 2 by the above it is false. For odd numbers three and greater the answer is not known.

 **Klas Markström** · 4 June 2009

I have modified my program to test the HOC for larger k as well. For k=6 it found counterexamples for n=2,3 and it is working on the optimal value for n=4 now.

For k=5 it has verified the conjecture for n=2,3,4,5, and it might be able to finish the computation for n=6 as well but it is very slow.

I'm a bit short on time today but I'll post the values for Fujimura for k=5,6 later, and compile a list of optimal solutions for the weighted problem for k=5,6 and small n as well.

 **Kristal Cantwell** · 4 June 2009

I am still working on the Pareto minimal counterexamples . I have done the first 6. They are in the wiki at the end of the section n=3.

 **Christian Elsholtz** · 5 June 2009

I have updated the section on lower bounds for Moser sets. In particular I have given a description what the bounds of Chvatal give. In fact, for dimension n>7 they improve our bounds in the spreadsheet.

I also updated some diacritics in the references.
Komlos to Koml\'{o}s and Chvatal to Chv\'{a}tal

To be done:
equation (1.1) should be changed in the light of the updates above.
For example by equation (2.2)

reference to Andries Brouwer's table or another explicit table of coding results on
A(n,d) into references.

Is any result known that states that the maximal size of a binary code wit n bits minimum distance d, i.e. A(n,d) is even (if it is not equal to 1)?

———
Kareem in case you read this: you recently mentioned you could do an extended computation. If I remember it right my suggestion of improving the programme (post 1116) was never taken up. I would hope that such a modification would save a lot of computational time.

 **Kareem Carr** · 5 June 2009

Dear Christian,

Good point about the modification you suggested. I was favoring something like distributing excutable copies of the program because it doesn't take too much time on my part (less than an hour) and might make a difference.

Your modification, while worthwhile, would probably take a day (at the most optimistic) or more to implement. The algorithm isn't conceptually complicated but doesn't mesh well with my data structures and my general program design.

I was intending to implement your idea a few months ago but the demands of classes/life in general got too high to put much time toward this project.

I have a little more time now. (Although due to some complications with moving, I haven't had access to my computer/programs/data for several days now). After several days of missed promises, according to the movers, I should see it today though! So I am living out a suitcase and doing all the other moving related stuff.

Terry's idea also looks like a productive direction but would require a full redesign from the ground up.

I am trying to navigate the time cost/benefit curves as best I can. Sorry about the delays.

Kareem

 **Kristal Cantwell** · 5 June 2009

I have the first 8 of the 16 Pareto minimal counterexamples.

 **Michael Peake** · 6 June 2009

I have written a Matlab script to calculate the lookup table for the [3]^4 brute-force calculations. I ran 1000 randomly chosen forbidden sets; it found an average of eight Pareto maxima for each forbidden set, with a maximum around 22 maxima.

It takes about four seconds for each forbidden set, although the empty forbidden set took thirty seconds. At this rate, it would take four months to run the complete table

 **Michael Peake** · 6 June 2009

My routine would now run in about two weeks, not four months.
There are 499 possible statistics that a Pareto line-free subset of [3]^3 can take, so with 2^27/48 lists, average eight Pareto sets, it might fit in 20MB or 40MB.